

Name :- Raunak Sapariya

Registration Number :- 26MCA0052

Problem 1 -> Take 2 stack called odd and even. Get inputs from the users. If input is odd then push it into the odd stack. If input is even then push it into the even stack.

CODE

```
package main

import "fmt"

func main() {

    var n int

    fmt.Print("Enter an Nnumber of integer: ")

    _, err := fmt.Scan(&n)
    if err != nil {
        fmt.Println("Error: Invalid input! Please enter an integer.")
    }

    even := make([]int,0)
    odd := make([]int,0)

    for i := 0; i < n; i++){
        var n int
        fmt.Printf("Enter a number %v: ",i+1)
        _, err = fmt.Scan(&n)
        if err != nil {
            fmt.Println("Error: Invalid input! Please enter an integer.")
            i --
        }

        if n%2 == 0 {
            even = append(even, n)
        } else {
            odd = append(odd, n)
        }
    }

    fmt.Println("Even",even)
    fmt.Println("Odd",odd)
}
```

OUTPUT

```
Enter an Number of integer: 5
Enter a number 1: 1
Enter a number 2: 2
Enter a number 3: 3
Enter a number 4: 4
Enter a number 5: 5
Even [2 4]
Odd [1 3 5]
```

Problem 2 -> Take 2 stack called vowels and consonants. Get inputs from the users. If input is Consonant then push it into the Consonant stack. If input is vowels, then push it into the vowels stack.

CODE

```
package main

import (
    "fmt"
)

func main() {
    var n int

    fmt.Print("Enter number of characters: ")

    _, err := fmt.Scan(&n)
    if err != nil {
        fmt.Println("Error: Please enter a valid integer.")
        return
    }

    var vowels []string
    var consonants []string

    for i := 0; i < n; i++ {
        var char string

        fmt.Printf("Enter character %d: ", i+1)

        _, err = fmt.Scan(&char)
        if err != nil {
            fmt.Println("Error: Invalid input.")
            i--
            continue
        }
    }
}
```

```

if len(char) != 1 {
    fmt.Println("Error: Please enter only one character.")
    i--
    continue
}

c := char[0]
if !((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
    fmt.Println("Error: Please enter an alphabet only.")
    i--
    continue
}

switch char {
case "a", "e", "i", "o", "u", "A", "E", "I", "O", "U":
    vowels = append(vowels, char)
default:
    consonants = append(consonants, char)
}
}

fmt.Println("\nVowels:", vowels)
fmt.Println("Consonants:", consonants)
}

```

OUTPUT

```

PS E:\VIT_DSA> go run .\vowel_const.go
Enter number of characters: 5
Enter character 1: a
Enter character 2: b
Enter character 3: c
Enter character 4: d
Enter character 5: e

Vowels: [a e]
Consonants: [b c d]

```

Problem 3 -> Write a program to get Array elements from 2-D array. Then rotate the elements of ROWS of array based on no. count .

CODE

```

package main

import "fmt"

```

```

func displayMatrix(arr [][]int, rows, cols int) {
    fmt.Println("\nArray after rotation:")
    for i := 0; i < rows; i++ {
        for j := 0; j < cols; j++ {
            fmt.Printf("%d ", arr[i][j])
        }
        fmt.Println()
    }
}

func main() {
    var rows, cols int

    fmt.Print("Enter number of rows: ")
    fmt.Scan(&rows)

    fmt.Print("Enter number of columns: ")
    fmt.Scan(&cols)

    arr := make([][]int, rows)
    for i := 0; i < rows; i++ {
        arr[i] = make([]int, cols)
    }

    fmt.Println("\nEnter array elements:")
    for i := 0; i < rows; i++ {
        for j := 0; j < cols; j++ {
            fmt.Printf("Enter element [%d][%d]: ", i, j)
            fmt.Scan(&arr[i][j])
        }
    }

    fmt.Println("\nOriginal array:")
    for i := 0; i < rows; i++ {
        for j := 0; j < cols; j++ {
            fmt.Printf("%d ", arr[i][j])
        }
        fmt.Println()
    }

    var count int
    fmt.Print("\nEnter rotation count: ")
    fmt.Scan(&count)

    count = count % cols

    for i := 0; i < rows; i++ {

        for k := 0; k < count; k++ {

```

```

    last := arr[i][cols-1]

    for j := cols - 1; j > 0; j-- {
        arr[i][j] = arr[i][j-1]
    }

    arr[i][0] = last
}

displayMatrix(arr, rows, cols)
}

```

OUTPUT

```

PS E:\VIT_DSA> go run .\rotate_row.go
Enter number of rows: 3
Enter number of columns: 3

Enter array elements:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [0][2]: 3
Enter element [1][0]: 4
Enter element [1][1]: 5
Enter element [1][2]: 6
Enter element [2][0]: 7
Enter element [2][1]: 8
Enter element [2][2]: 9

Original array:
1 2 3
4 5 6
7 8 9

Enter rotation count: 3

Array after rotation:
1 2 3
4 5 6
7 8 9
PS E:\VIT_DSA>

```

```
PS E:\VIT_DSA> go run .\rotate_row.go
```

```
Enter number of rows: 3
```

```
Enter number of columns: 3
```

```
Enter array elements:
```

```
Enter element [0][0]: 1
```

```
Enter element [0][1]: 2
```

```
Enter element [0][2]: 3
```

```
Enter element [1][0]: 4
```

```
Enter element [1][1]: 5
```

```
Enter element [1][2]: 6
```

```
Enter element [2][0]: 7
```

```
Enter element [2][1]: 8
```

```
Enter element [2][2]: 9
```

```
Original array:
```

```
1 2 3
```

```
4 5 6
```

```
7 8 9
```

```
Enter rotation count: 5
```

```
Array after rotation:
```

```
2 3 1
```

```
5 6 4
```

```
8 9 7
```

Problem 4 -> Write a program to get Array elements from 2-D array. Then rotate the elements of COLUMNS of array based on no. count.

CODE

```
package main
```

```
import "fmt"
```

```
func displayMatrix(arr [][]int, rows, cols int) {
```

```
    fmt.Println("\nArray after rotation:")
```

```
    for i := 0; i < rows; i++ {
```

```
        for j := 0; j < cols; j++ {
```

```
            fmt.Printf("%d ", arr[i][j])
```

```
        }
```

```
        fmt.Println()
```

```
    }
```

```
}
```

```
func main() {
```

```
    var rows, cols int
```

```

fmt.Print("Enter number of rows: ")
fmt.Scan(&rows)

fmt.Print("Enter number of columns: ")
fmt.Scan(&cols)

arr := make([][]int, rows)
for i := 0; i < rows; i++ {
    arr[i] = make([]int, cols)
}

fmt.Println("\nEnter array elements:")
for i := 0; i < rows; i++ {
    for j := 0; j < cols; j++ {
        fmt.Printf("Enter element [%d][%d]: ", i, j)
        fmt.Scan(&arr[i][j])
    }
}

fmt.Println("\nOriginal array:")
for i := 0; i < rows; i++ {
    for j := 0; j < cols; j++ {
        fmt.Printf("%d ", arr[i][j])
    }
    fmt.Println()
}

var count int
fmt.Print("\nEnter rotation count: ")
fmt.Scan(&count)

count = count % cols

for j := 0; j < cols; j++ {

    for k := 0; k < count; k++{

        last := arr[rows-1][j]

        for i := rows - 1; i > 0; i--{
            arr[i][j] = arr[i-1][j]
        }

        arr[0][j] = last
    }

}

displayMatrix(arr, rows, cols)

```

```
}
```

OUTPUT

```
PS E:\VIT_DSA> go run .\rotate_column.go
Enter number of rows: 3
Enter number of columns: 3

Enter array elements:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [0][2]: 3
Enter element [1][0]: 4
Enter element [1][1]: 5
Enter element [1][2]: 6
Enter element [2][0]: 7
Enter element [2][1]: 8
Enter element [2][2]: 9

Original array:
1 2 3
4 5 6
7 8 9

Enter rotation count: 3

Array after rotation:
1 2 3
4 5 6
7 8 9
PS E:\VIT_DSA> █
```

```
PS E:\VIT_DSA> go run .\rotate_column.go
```

```
Enter number of rows: 3
```

```
Enter number of columns: 3
```

```
Enter array elements:
```

```
Enter element [0][0]: 1
```

```
Enter element [0][1]: 2
```

```
Enter element [0][2]: 3
```

```
Enter element [1][0]: 4
```

```
Enter element [1][1]: 5
```

```
Enter element [1][2]: 6
```

```
Enter element [2][0]: 7
```

```
Enter element [2][1]: 8
```

```
Enter element [2][2]: 9
```

```
Original array:
```

```
1 2 3
```

```
4 5 6
```

```
7 8 9
```

```
Enter rotation count: 5
```

```
Array after rotation:
```

```
4 5 6
```

```
7 8 9
```

```
1 2 3
```

```
PS E:\VIT_DSA> █
```

Problem 5 -> 5. Get 1-D array then insert element in specific position and delete from specific position.

CODE

```
package main

import (
    "fmt"
)

func main() {
    var n int
    fmt.Print("Enter the size of array: ")
    fmt.Scan(&n)

    arr := make([]int, n)

    fmt.Println("Enter the elements of the array:")
    for i := 0; i < n; i++ {
        fmt.Printf("Element %d: ", i+1)
        fmt.Scan(&arr[i])
    }
}
```

```

}
fmt.Println("\nOriginal array:", arr)

// Insert
var position, value int
fmt.Printf("\nEnter the position to insert the element (0 to %d): ", len(arr)-1)
fmt.Scan(&position)

if position < 0 || position >= len(arr) {
    fmt.Println("Invalid position!")
    return
}

fmt.Print("Enter the value to insert: ")
fmt.Scan(&value)

arr = append(arr[:position], append([]int{value}, arr[position:]...)...)

fmt.Println("Array after insertion:", arr)

// Delete
fmt.Printf("\nEnter the position to delete the element (0 to %d): ", len(arr)-1)
fmt.Scan(&position)

if position < 0 || position >= len(arr) {
    fmt.Println("Invalid position!")
    return
}

arr = append(arr[:position], arr[position+1:]...)
fmt.Println("Array after deletion:", arr)
}

```

OUTPUT

```
PS E:\VIT_DSA> go run .\1d.go
Enter the size of array: 5
Enter the elements of the array:
Element 1: 1
Element 2: 2
Element 3: 3
Element 4: 4
Element 5: 5

Original array: [1 2 3 4 5]

Enter the position to insert the element (0 to 4): 4
Enter the value to insert: 9
Array after insertion: [1 2 3 4 9 5]

Enter the position to delete the element (0 to 5): 3
Array after deletion: [1 2 3 9 5]
```